

Introduction to JWT and Spring Boot

What is a JWT?

- A compact, URL-safe method for securely transmitting information between parties as a JSON object.
- A JSON Web Token is an open standard (RFC 7519) for securely transmitting information as a JSON object between two parties.
- **Key features:**
 - **Self-contained:** The token itself contains the necessary user information.
 - **Compact:** Small enough to be sent in URL parameters or HTTP headers.
 - **Stateless:** Eliminates the need for a server to maintain session state.
 - **Signed:** Digitally signed to verify its authenticity and integrity.
-

The JWT structure

- Anatomy: A JWT is a single string with three parts separated by dots (.).
- Visual: A large, colorful diagram highlighting the three parts:
 - Header(e.g., in red)
 - Payload (e.g., in purple)
 - Signature (e.g., in blue)

The Header

- Purpose: Contains metadata about the token.
- Contents:
 - "typ": "JWT" (type of token)
 - "alg": "HS256" (hashing algorithm used for the signature)
- Process: This JSON is Base64Url encoded to form the first part of the token string.
- Visual: Show the JSON object { "alg": "HS256", "typ": "JWT" } next to its encoded string equivalent.

The Payload (Claims)

- Purpose: Contains the "claims," or statements about an entity (like a user).
- Types of claims:
 - Registered claims: Standardized, optional claims like iss (issuer), exp (expiration time), and sub (subject).
 - Private claims: Custom claims agreed upon by the communicating parties.
- Important note: The payload is only encoded, not encrypted. Anyone can decode and read it, so you should not store sensitive information here.
- Visual: Show an example JSON object, such as { "sub": "1234567890", "name": "John Doe", "admin": true, "exp": 1516239022 }, next to its encoded string.

The Signature

- Purpose: Ensures the token has not been tampered with.
- Creation process:
 - Take the encoded header and encoded payload.
 - Combine them with a secret key.
 - Hash them using the algorithm specified in the header.
- Visual: Use a flowchart to illustrate the process:
 - Encoded Header + "." + Encoded Payload
 - Use alg with Secret key --> Create Signature

How JWTs work (The flow)

- Step 1: Login: The user sends credentials to the server.
- Step 2: Token generation: The server verifies credentials and, if correct, creates and sends a JWT back to the client.
- Step 3: Storing the token: The client (e.g., browser) stores the JWT.
- Step 4: Accessing resources: The client sends the JWT with subsequent requests, typically in the Authorization header.
- Step 5: Verification: The server validates the token's signature, and if it's valid, grants access.
- Visual: A sequence diagram showing the interactions between the client, the server, and the JWT.

Advantages of JWT

- Stateless: The server does not need to store session data, which improves scalability.
- Compact: Smaller than older standards like SAML, making them easy to transmit.
- Decentralized: A single sign-on provider can issue a JWT that can be verified across different applications.
- Cross-domain friendly: JWTs work seamlessly across microservices and different platforms, including web and mobile.
- Visual: Use icons representing key benefits, such as a cloud for scalability and a checkmark for security.

Security Best Practices

- Use HTTPS: Always transmit JWTs over a secure connection.
- Set expiration (exp): Use short-lived tokens to reduce the window of exposure if a token is stolen.
- Secure storage: Store JWTs in a secure, server-side accessible way, such as HttpOnly cookies, instead of local storage.
- Keep secrets safe: Store the signing key securely on the server.
- Revocation: Implement a method for blacklisting or revoking tokens before their expiration date.
- Visual: A checklist of best practices with green checkmarks and clear, concise points.

Why use JWT with Spring Boot?

- To build secure, stateless REST APIs that can scale easily.
- for securing RESTful APIs and building scalable, distributed systems
- **Stateless Authentication:**
 - JWTs enable stateless authentication. Unlike traditional session-based authentication where the server maintains session state, JWTs are self-contained and carry all necessary user information.
 - This eliminates the need for server-side session storage, making your application more scalable and efficient, especially in microservices architectures or when dealing with load balancing.
- **Scalability and Performance**
 - The stateless nature of JWTs improves scalability as requests can be handled by any server instance without needing to retrieve session information from a central store. This reduces server load and enhances performance.
- **Security**
- **Integrity:** JWTs are signed (using a secret key or public/private key pair), ensuring that their contents have not been tampered with during transit.
- **Authentication and Authorization:** After a successful login, a JWT is issued to the client. This token is then included in subsequent requests to authenticate the user and determine their authorization to access specific resources.
- **Cross-Domain and Microservices Compatibility:**
 - JWTs are well-suited for cross-domain authentication and communication between different microservices
- **Reduced Database Lookups:**
 - Once a JWT is issued and validated, subsequent requests can often be authorized based on the token itself, reducing the need for frequent database lookups to retrieve user roles or permissions.

Project Setup

Create a Spring Boot project and include dependencies for Spring Security and JWT (Java JWT).

Code

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.5</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
```

SecurityConfig.java

@Configuration

```
public class SecurityConfig {
```

@Autowired

```
private JwtAuthFilter authFilter;
```

@Autowired

```
UserDetailsService userDetailsService;
```

@Bean

```
public AuthenticationProvider authenticationProvider(){
```

```
    DaoAuthenticationProvider provider=new DaoAuthenticationProvider();
```

```
    provider.setUserDetailsService(userDetailsService);
```

```
    //provider.setPasswordEncoder(new BCryptPasswordEncoder(12));
```

```
    provider.setPasswordEncoder(passwordEncoder());
```

```
    return provider;
```

```
}
```

@Bean

```
public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws  
Exception {
```

```
    return config.getAuthenticationManager();
```

```
}
```

@Bean

```
public SecurityFilterChain securityFilterChain(HttpSecurity  
security) throws Exception {
```

```
    security.csrf(c->c.disable());
```

```
        authorizeHttpRequests(req-  
>req.requestMatchers("/addNewUser",  
"/generateToken").permitAll());
```

```
        authorizeHttpRequests(req-  
>req.requestMatchers("/welcome").authenticated());
```

```
        //authorizeHttpRequests(req-  
>req.anyRequest().authenticated());
```

```
httpBasic(Customizer.withDefaults()).sessionManagement(sessi  
on-
```

```
>session.sessionCreationPolicy(SessionCreationPolicy.STATELE  
SS)).authenticationProvider(authenticationProvider()).addFilterB  
efore(authFilter, UsernamePasswordAuthenticationFilter.class);
```

```
    return security.build();
```

```
}
```

@Bean

```
public PasswordEncoder passwordEncoder() {
```

```
    return new BCryptPasswordEncoder();
```

```
}
```

JwtAuthFilter

// This class helps us to validate the generated jwt token

@Component

@Slf4j

public class JwtAuthFilter extends OncePerRequestFilter {

 @Autowired

 private JwtService jwtService;

 @Autowired

 private UserService userDetailsService;

 @Override

 protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain) throws ServletException, IOException {

 String authHeader = request.getHeader("Authorization");

 String token = null;

 String username = null;

 if (authHeader != null && authHeader.startsWith("Bearer ")) {

 token = authHeader.substring(7);

 username = jwtService.extractUsername(token);

 }

 if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {

 UserDetails userDetails = userDetailsService.loadUserByUsername(username);

 if (jwtService.validateToken(token, userDetails)) {

 UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuthorities());

 authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

 SecurityContextHolder.getContext().setAuthentication(authToken);

 }

 }

 filterChain.doFilter(request, response);

 }

}

Model/dto

@Data

@AllArgsConstructor

@NoArgsConstructor

```
public class AuthRequest {
```

```
    private String username;
```

```
    private String password;
```

```
}
```

@Data

@Entity

@Table(name = "Users")

```
public class User {
```

```
    @Id
```

```
    @GeneratedValue
```

```
    private int id;
```

```
    private String name;
```

```
    private String email;
```

```
    private String password;
```

```
    private String roles;
```

```
}
```

UserPrincipal

```
public class UserPrincipal implements UserDetails {

    private String name;

    private String password;

    private List<GrantedAuthority> authorities;

    public UserPrincipal(User user) {
name = user.getName();

        password = user.getPassword();

        authorities = Arrays.stream(user.getRoles().split(",")).map(SimpleGrantedAuthority::new)

            .collect(Collectors.toList());
    }

    @Override

    public Collection<? extends GrantedAuthority> getAuthorities() {

        //return Collections.singleton(new SimpleGrantedAuthority("ADMIN"));

        return authorities;
    }

    @Override

    public String getPassword() {

        return password;
    }
}
```

```
@Override

    public String getUsername() {

        return name;
    }

    @Override

    public boolean isAccountNonExpired() {

        return true;
    }

    @Override

    public boolean isAccountNonLocked() {

        return true;
    }

    @Override

    public boolean isCredentialsNonExpired() {

        return true;
    }

    @Override

    public boolean isEnabled() {

        return true;
    }
}
```

@Repository

public interface UserRepository

extends

JpaRepository<User,Integer> {

Optional<User>

findByName(String name);

}

jwt-service

@Slf4j

@Component

```
public class JwtService {

    public static final String SECRET =
"5367566B59703373367639792F423F4528482B4D6251655468576D5A7134
7437";

    public String generateToken(String userName) {

        Map<String, Object> claims = new
HashMap<>();

        log.warn("createToken is called..");

        return createToken(claims, userName);

    }

    private String createToken(Map<String, Object> claims, String
userName) {

        return Jwts.builder()

            .setClaims(claims)

            .setSubject(userName)

            .setIssuedAt(new
Date(System.currentTimeMillis()))

            .setExpiration(new
Date(System.currentTimeMillis() + 1000 * 60 * 30))

            .signWith(getSignKey(),
SignatureAlgorithm.HS256).compact();

    }
}
```

```
private Key getSignKey() {
```

```
        byte[] keyBytes=
Decoders.BASE64.decode(SECRET);

        return Keys.hmacShaKeyFor(keyBytes);

    }
```

```
public String extractUsername(String token) {

        return extractClaim(token,
Claims::getSubject);

    }

    public Date extractExpiration(String token) {

        return extractClaim(token,
Claims::getExpiration);

    }
```

```
    public <T> T extractClaim(String token,
Function<Claims, T> claimsResolver) {

        final Claims claims =
extractAllClaims(token);

        return claimsResolver.apply(claims);

    }
```

```

        private Claims extractAllClaims(String token) {
            return Jwts
                .parserBuilder()
                .setSigningKey(getSignKey())
                .build()
                .parseClaimsJws(token)
                .getBody();
        }

        private Boolean isTokenExpired(String token) {
            return
            extractExpiration(token).before(new Date());
        }

        public Boolean validateToken(String token,
            UserDetails userDetails) {
            final String username =
            extractUsername(token);
            return
            (username.equals(userDetails.getUsername()) &&
            !isTokenExpired(token));
        }
    }
}

```

userservice

@Service

```
public class UserService implements UserDetailsService {
```

```
    @Autowired
```

```
    private UserRepository userRepository;
```

```
    @Override
```

```
    public UserDetails loadUserByUsername(String username)  
    throws UsernameNotFoundException {
```

```
        Optional<User> byUserName =  
        userRepository.findByName(username);
```

```
        if(byUserName.isEmpty())
```

```
            System.out.println("User Not present");
```

```
        return new UserPrincipal(byUserName.get());
```

```
    }
```

```
    public String addUser(User userInfo) {
```

```
        userInfo.setPassword(new  
        BCryptPasswordEncoder(12).encode(userInfo.getPassword()  
        ));
```

```
        userRepository.save(userInfo);
```

```
        return "User Added Successfully";
```

```
    }
```

```
}
```

personcontroller

@Slf4j

@RestController

public class PersonController {

 @Autowired

 private AuthenticationManager authenticationManager;

 @Autowired

 private UserService service;

 @Autowired

 private JwtService jwtService;

 @GetMapping("/welcome")

 public String hello() {

 return "hello world";

 }

 @PostMapping("/generateToken")

 public String authenticateAndGetToken(@RequestBody
 AuthRequest authRequest) {

 Authentication authentication =
 authenticationManager.authenticate(new
 UsernamePasswordAuthenticationToken(authRequest.getUsername(),
 authRequest.getPassword()));

 if (authentication.isAuthenticated()) {

 log.warn("authenticated");

 return
 jwtService.generateToken(authRequest.getUsername());

 } else {

 log.error("invalid user request");

 throw new UsernameNotFoundException("invalid user
request!");

 }

 }

 @PostMapping("/addNewUser")

 public String addNewUser(@RequestBody User userInfo) {

 return service.addUser(userInfo);

 }

}

Main and running

```
package com.abc;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;
ication;

@SpringBootApplication
public class SpringSecurityApp1Application {

    public static void main(String[] args) {

        SpringApplication.run(SpringSecurityApp1App
lication.class, args);

    }

}
```

```
post
http://localhost:8080/addNewUser
{
    "name":"suresh kumar",
    "email":"suresh@ymail.com",
    "password":"1234",
    "roles":"ROLE_USER"
}
post
http://localhost:8080/generateToken
{
    "username":"suresh kumar",
    "password":"1234"
}
```